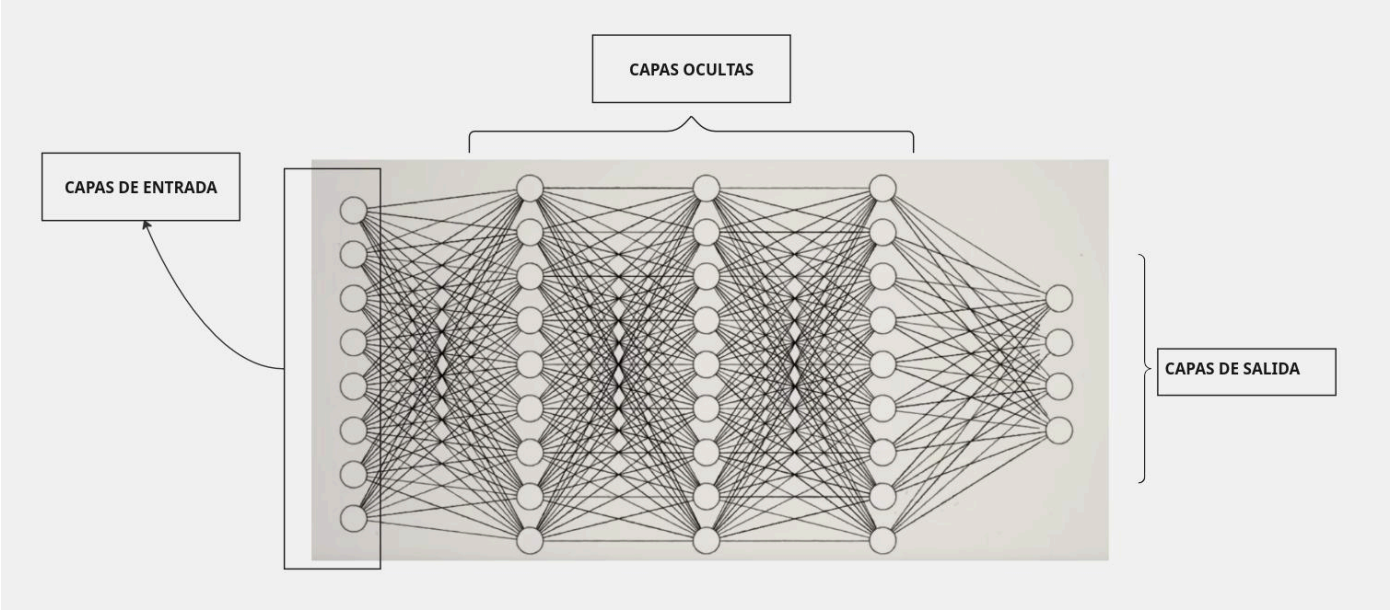
# REDES NEURONALES

Una red neuronal artificial es un modelo matemático inspirado en el funcionamiento del cerebro humano. Está diseñado para **aprender patrones a partir de datos** y tomar decisiones o hacer predicciones. En otras palabras, se trata de una familia de algoritmos muy potentes con los que podemos modelar comportamientos inteligentes.

**Estructura básica:**

- **Capa de entrada:** Reciben los datos (ej: pixeles de una imagen, números).
- **Capas ocultas:** Hacen cálculos y transformaciones intermedias.
- **Capas de salida:** Devuelve la predicción final (ej: “gato”, “no fraude”, “positivo”).

Cada conexión tiene un peso que indica su importancia, y cada nodo aplica una función de activación (ej: ReLU, sigmoid, etc.)




**A todo esto, ¿qué es una neurona?**
Una neurona es una unidad computacional que realiza una operación matemática para calcular una suma ponderada de entrada a la que le aplica un **bias** y luego una **función de activación.** Es parte de una **RED NEURONAL** y tiene **parámetros entrenables: PESOS Y BIAS.**

**Suma Ponderada: En lugar de sumar los valores directamente, cada valor se multiplica por un número (el peso) antes de sumarlos. Es una suma de valores donde cada uno tiene un "peso" que indica su importancia.**

Una neurona toma:

- Entradas: X1, X3, X4, ….,Xn
- Pesos: W1, W2, W3, …., Wn.
- Un sesgo o bias: b

Y calcula utilizando una función como:

- **z = w1 . x1 + w2 . x2 +⋯+ wn . xn + b**

Este valor 'z' obtenido, luego pasa por una **función de activación** (como ReLU, Sigmoid, etc) para producir una salida.

- salida = $\phi(z)$ donde '$\phi$' es la funcion de activacion.

**Ejemplo:**

Pensemos en un caso donde tengamos una neurona con 2 entradas, Horas de estudio y Clases asistidas. Para este ejemplo, pensemos que tenemos en total 5 horas de estudio y que lo nombramos como X1, entonces X1 = 5. Para Clases asistidas diremos que el alumno asistió a 8 clases, este será X2, entonces X2 = 8.

Recordemos que los pesos, lo obtenemos durante el entrenamiento, pero en este caso, diremos que los pesos son:

- W1 = 0.3
- W2 = 0.6
- Bias = -2

Nuestra neurona utiliza la siguiente función:

- $z = 0.3 \cdot 5 + 0.6 \cdot 8 - 2 = 1.5 + 4.8 - 2 = 4.3$

Si usamos una función sigmoide como activación tendríamos el siguiente resultado:

- salida = $1/(1 + e^{-4.3}) = 0.986$

El resultado es casi 1, lo que podría interpretarse como una predicción de que aprobara con alta probabilidad.

A todo esto, podríamos decir que una neurona es como una función que tiene un estado que va cambiando constantemente. Estas neuronas, poseen estructuras de datos llamadas '**TENSORES**'.

**TENSORES**

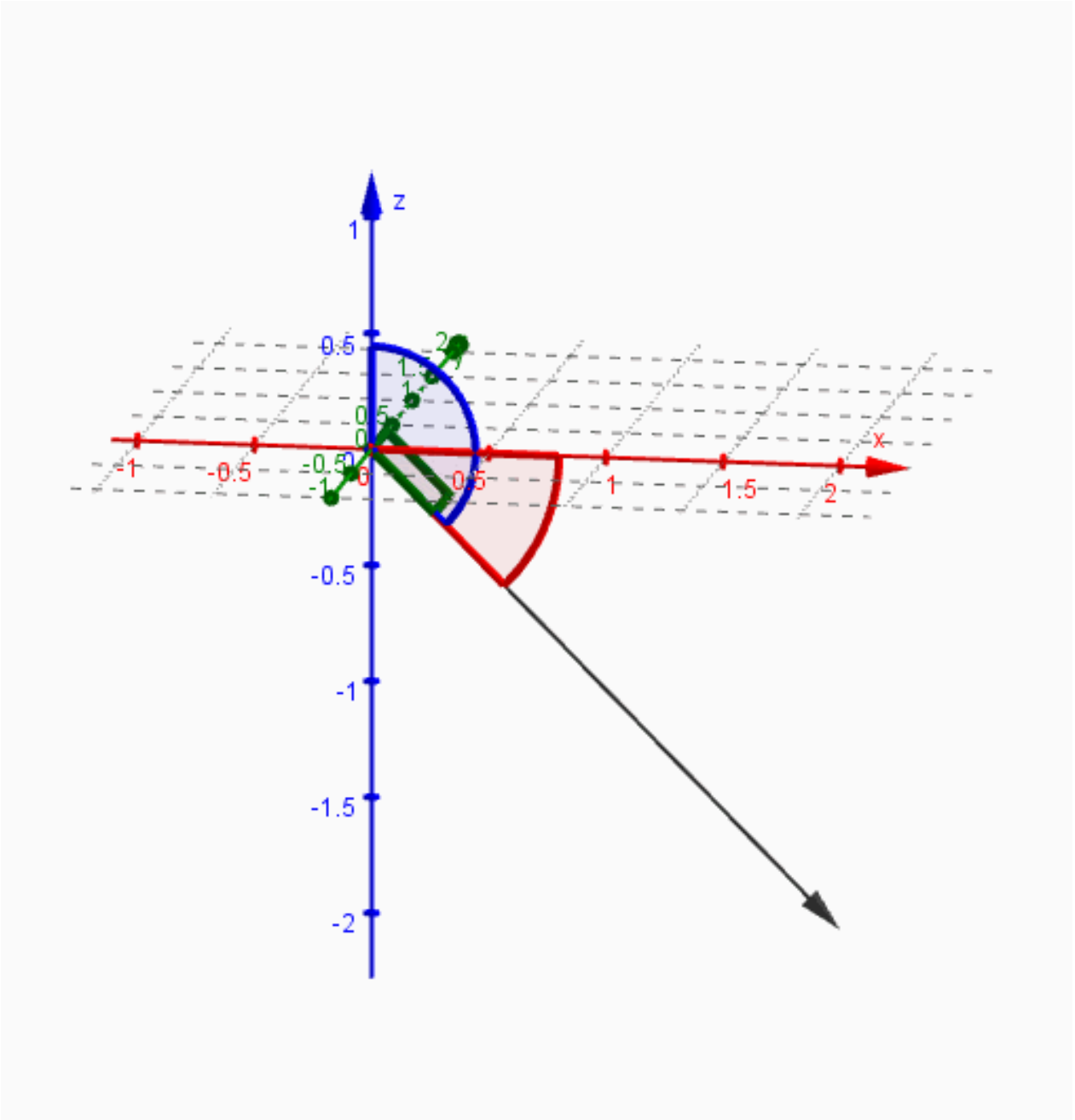
Es una estructura de datos matemática que generaliza los conceptos de escalares, vectores y matrices a cualquier número de dimensiones. Es la unidad fundamental para representar y manipular datos en machine learning y deep learning, especialmente en librerías como Pytorch, TensorFlow y Numpy.

**Diferencias entre las estructuras de datos**

| TIPO DE DATO | NOMBRE TÉCNICO | EJEMPLO | DIMENSIÓN |
|---|---|---|---|
| Número | Escalar | 7 | 0D |
| Lista | Vector | [1,2,3,5,4] | 1D |
| TABLA | Matriz | [[1,2],[3,4]] | 2D |
| Cubo de datos | Tensor | [[[...]]] (imagen RGB, por ejemplo) | 3D |
| Más dimensiones | Tensor | Usado en videos, lotes de datos, etc. | 4D+ |

Entonces, podemos decir que un Tensor es un **objeto multidimensional** que puede ser interpretado como una **función multilineal** entre espacios vectoriales. En deep learning, se representa como un arreglo de números de N dimensiones, que permite operar sobre grandes volúmenes de datos de forma eficiente.

**EJEMPLO DE UN TENSOR DE VARIAS DIMENSIONES.**



# Mecanismos y funciones claves de una red neuronal

## Funciones de activación

Una función de activación es una función matemática que decide **si una neurona se activa o no,** es decir, cuanto de la información que recibe una neurona debe pasar a la siguiente capa. Sin una función de activación, la red neuronal sería solo una gran combinación lineal de números. Eso significa que **no podría aprender relaciones complejas ni resolver problemas como clasificación no lineal.**

**Es como darle a la red la capacidad de decir:**

- **"📢 ¡Esto me interesa! (activo fuerte)"**
- **"🤔 Meh, esto no importa tanto (activo débil)"**
- **"🛑 Esto no lo paso (me inhibo)".**

**¿Cómo funciona?**
Cada neurona calcula:

- $z = w1 \,.\, x1 + w2 \,.\, x2 + \cdots + wn \,.\, xn + b$

Y luego le aplica la función de activación:

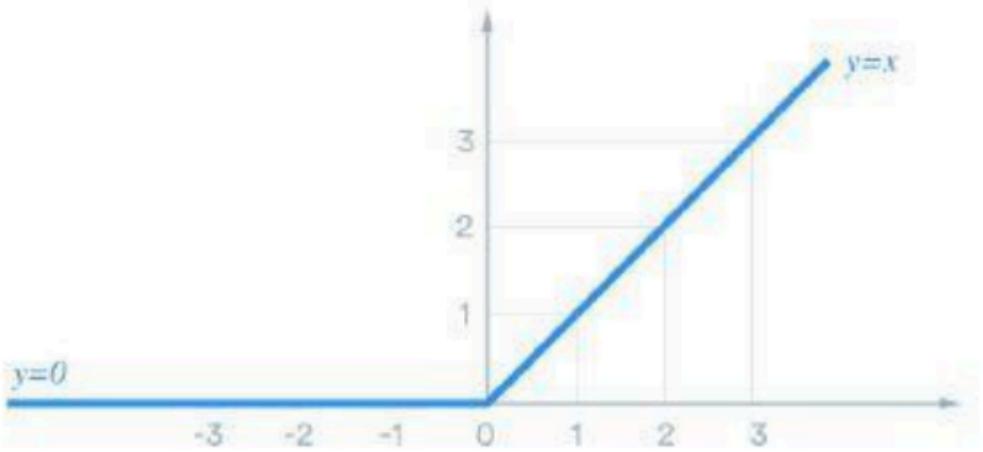
- salida = ϕ(z)

**Ejemplos de las funciones de activación más utilizadas.**

ReLU (Rectified Linear Unit)

- Es simple y rapida.
- Se utiliza mucho en redes neuronales profundas.
- Si el valor es negativo, lo "apaga" (0), si es positivo lo deha pasar.
- Su gráfica es como una rampa que arranca en cero y crece hacia la derecha.

$$f(x) = max(0, x)$$




2 **Sigmoid(Sigmoide)**

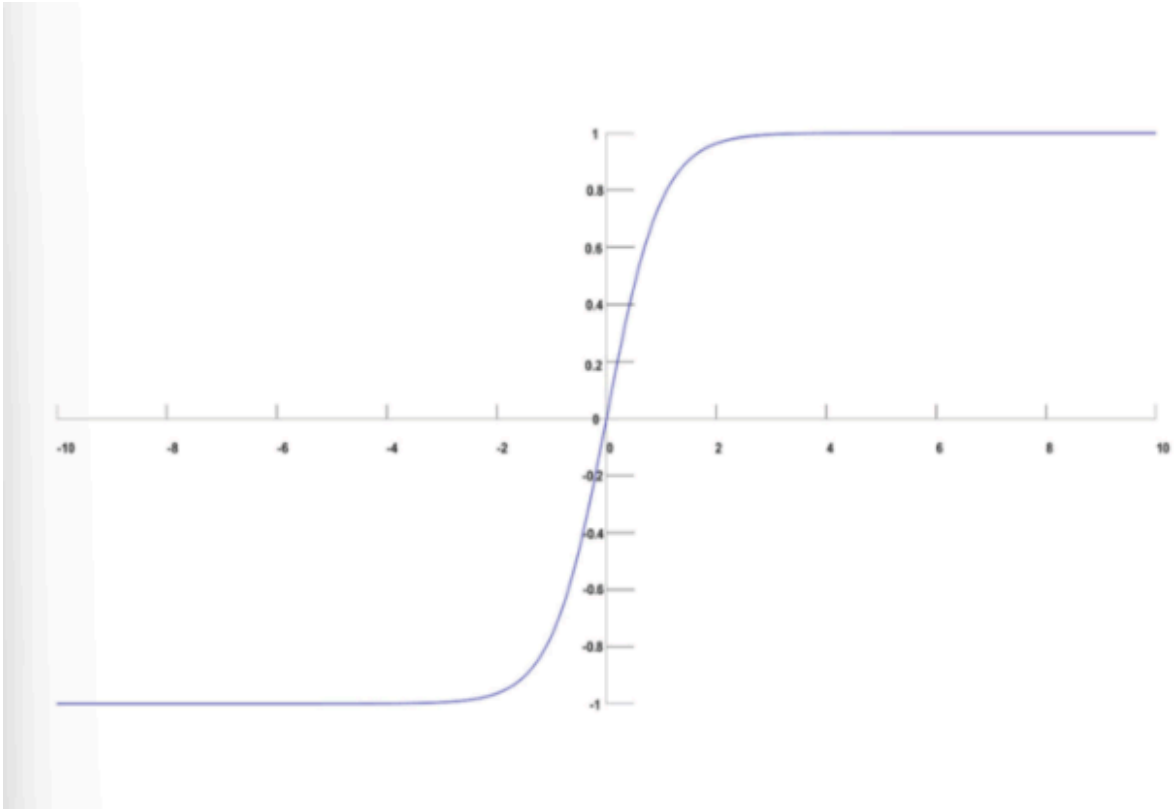
- Su salida siempre está entre 0 y 1.
- Es muy utilizada para **clasificación binaria.**
- Su gráfica es una forma de "**S**", aplana valores extremos y resalta intermedios.

$$f(x) = \frac{1}{1 + e^{-x}}$$




**3. SoftMax**

La función de activacion softmax se utiliza en la capa de salida de una red neuronal para realizar la clasificacion multiclase. Esta funcion se utiliza para convertir las saluidas de la capa anteior en probabilidades que suman uno. Las probabilidades se utilizan para medir la confianza del modelo en la pertenencia a cada clase.




**Forward Pass o Forward Propagation**

Es la fase en la que una red neuronal procesa una entrada y genera una salida. Es el “camino hacia adelante” desde la capa de entrada hasta la capa de salida.

Paso a paso:

Supongamos que una entrada X pasa por una red con una capa oculta y una capa de salida.

1. Cada neurona toma la entrada **x**, la multiplica por un peso **w**, le suma un sesgo **b**.

$$z = w \cdot x + b$$

2. El valor **z** se pasa por una **función de activación** para introducir **no linealidad:**

$$\mathbf{a = f(z)}$$

Esto se repite capa por capa hasta llegar a la salida final.

**Loss function (función de pérdida)**

Es una función matemática que mide **qué tan mal está prediciendo una red neuronal**. Es el corazón del proceso de entrenamiento, porque le dice a la red **cuánto se está equivocando**.

**¿Qué hace exactamente?**

**Compara la salida predicha** por la red (**ŷ)** con el valor **real (y)** y devuelve un número:

- Si el número es grande, la predicción es mala.
- Si el número es cero o pequeño, la predicción es buena.

Ese número es el **error**, y es lo que la red intenta **minimizar** durante el entrenamiento usando un algoritmo llamado **backpropagation.**

**Loss Functions más utilizadas:**

1. **MSE (**Error cuadrático medio**):**
   - Penaliza más los errores grandes.
   - Muy usada en modelos de regresión lineal, que predicen números(precios, temperatura, etc. ).

$$mse = \frac{1}{n}\sum(y_n - \bar{y})^2$$

2. **MAE (**Error medio absoluto**):**
   - Es más robusto frente a el error cuadrático medio, lo que significa que el entrenamiento del modelo se ve menos influenciado por datos anómalos que pueda haber en el conjunto de entrenamiento.
   - MAE no eleva al cuadrado las diferencias, lo que lo hace menos sensible a los valores atípicos.

$$MAE = \sum_{i=1}^{n} \frac{|\hat{y}_i - y_i|}{n}$$

**Binary Cross Entropy (Entropía cruzada binaria)**

La entropía cruzada binaria, es una medida de precisión para **variables binarias.**

- Más difícil diferenciación y convergencia.
- Escala univariante.
- Simétrica,
- Es más fácil de interpretar.

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} y_i \log(p_i + (1-y)\log(1-pi))$$

**Categorical Cross-Entropy (Entropía cruzada binaria)**

La entropía cruzada categórica, es una medida de precisión para **variables categóricas.**

**Características del Entropía cruzada categórica:**

- **Más difícil diferenciación y convergencia.**
- **Escala univariante.**
- **Simétrica.**
- **Es más fácil de interpretar.**

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m} y_{ij} \log(p_{ij})$$

**Backpropagation (Retropropagación del error)**

Es un algoritmo clave que permite a una red neuronal **aprender**. Sirve para ajustar los pesos y bias de forma que el error (**loss**) **disminuya en cada iteración del entrenamiento.**

Después de que la red hace una predicción con el forward pass, se compara con el valor real y se calcula un error (**loss**). Calcula **cuánto contribuyó cada peso al error** usando gradientes (derivadas) y luego, ajusta los pesos en dirección contraria al error para mejorar (descenso del gradiente).

**¿Cómo funciona?**

1. **Forward pass:**
   Se calcula la salida de la red usando los pesos actuales

2. **Loss:**
   Se calcula el error entre la salida predicha y el valor real.
3. **Retropropagación:**
   Se usan derivadas parciales para calcular el **gradiente** de la función de pérdida con respecto a cada peso. Se aplica la **regla de la cadena** porque los pesos afectan al error de forma indirecta a través de múltiples capas.

4. **Actualización de los pesos:**
   Con un **optimizador**, se actualizan los pesos y se vuelve a hacer la predicción.

**Optimizadores**

Los optimizadores son algoritmos que **ajustan los pesos y bias** de una red neuronal para **minimizar la función de pérdida (loss).** Son fundamentales en el entrenamiento porque **deciden cómo y cuándo modificar los parámetros** en base a los gradientes calculados durante la backpropagation.

**Tipos de optimizadores más utilizados**

1. **SGD (Descenso de gradiente estocástico):**
   Actualiza los pesos usando un pequeño grupo de datos (batch) por vez. Intenta encontrar mínimos y máximos por iteración. El gradiente indica la dirección en la que la función tiene el ratio de aumento más pronunciada aunque no indica hasta donde se debe avanzar en esa dirección.

   Su fórmula básica es:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n} Q_i(w),$$

2. **Momentum:**
   Mejora a SGD agregando una “inercia” para evitar que el optimizador quede atrapado en mínimos locales

3. **ADAM (Adaptive Moment Estimation):**
   El optimizador ADAM trata de solventar el problema con la fijación del ratio de aprendizaje del SGD, para ello adapta el ratio de aprendizaje en función de cómo estén distribuidos los parámetros. Si los parámetros están muy dispersos el ratio de aprendizaje aumentará. Este algoritmo combina las ventajas de otros dos algoritmos de optimización: Momentum y RMSprop, y se caracteriza por calcular tasas de aprendizaje adaptativas para cada parámetro basándose en estimaciones del primer y segundo momento de los gradientes

**Ciclo de entrenamiento de una Red Neuronal**

1. Inicializamos los pesos **aleatoriamente.**
2. Para cada lote de datos (batchs de entrenamiento) hacemos lo siguiente:
   - Fordward pass.
   - Calcula la perdida (función de perdida).
   - Backpropagation (retropropagación)
   - Se actualizan los pesos con el **optimizador.**
3. Se repite por varias epocas (epochs) hasta que el modelo converja.